

gRPC

COMS 4156: Advanced Software Engineering
Fall 2024

Presented by Justin Tan (jjt2163)

About Me

- Undergraduate CS major
- Originally from SF , arrived at Columbia in 2021
- 4 years of professional software engineering experience
 - Mostly fintech startups
 - Consumer-facing web applications
 - Distributed ETL pipelines
- Teaching qualifications: I've toilet-trained my three cats :)

Agenda

1. Introduction
2. Core Concepts
3. Live Demo
4. Closing
5. Resources

Introduction: What is gRPC?



- Stands for **gRPC Remote Procedure Calls***
 - Yes, it's recursive
- **High performance**, open-source RPC communication framework by Google
- **Alternative to REST** for building APIs and talking between services

* gRPC = great Ramen Places near Columbia???

Introduction: What is gRPC?

Traditional REST APIs

- **Simple**, widely adopted
- **Stateless** (each request is independent / self-contained)
- **Less efficient** for large payloads due to text-based format (typically JSON)

gRPC

- Leverages **HTTP/2** for efficient, fast communication
- Data is serialized with **Protocol Buffers** (more compact and type safe)
- Enables more sophisticated **client-server** communication

Introduction: Common Uses

Microservices

- Services are responsible for one business function (e.g., user management, payment processing)
- gRPC excels at **service-to-service communication**, especially in high-traffic environments



and many others...

Introduction: Common Uses

Mobile applications

- For messaging or location updates, polling at regular intervals increases traffic (even if there are no updates) and adds latency
- gRPC **bidirectional streaming** helps in real-time communication



and many others...

Core Concepts: Traditional REST API

The architectural style we all know and love (?)

GET /restaurants/116/entrees

- Resource-based, not function based
- Standard HTTP methods (GET, POST, etc.)

Example response:

A lot of **redundant data** is encoded in the response (e.g. `entree_id`, `name`, etc. n times)

Next, we will see how gRPC and **protocol buffers** can do this more efficiently.

```
{
  "restaurant_id": 116,
  "entrees": [
    {
      "entree_id": 1,
      "name": "Tonkotsu Ramen",
      "description": "Rich pork broth with tender noodles",
      "price": 12.99
    },
    {
      "entree_id": 2,
      "name": "Shoyu Ramen",
      "description": "Soy sauce-based broth with pork",
      "price": 11.99
    }
  ]
}
```

Core Concepts: Protocol Buffers (protobuf)

gRPC uses **protocol buffers** as a mechanism to serialize data into a binary format.

The **service and message definitions** are defined in a `.proto` file.

A **protobuf compiler** (`protoc`) generates the client and server code to handle these operations.

```
syntax = "proto3";

service RestaurantService {
    rpc GetEntrees (EntreeRequest) returns (EntreeResponse);
}

message EntreeRequest {
    int32 restaurant_id = 1;
}

message EntreeResponse {
    repeated Entree entrees = 1;
}

message Entree {
    int32 entree_id = 1;
    string name = 2;
    string description = 3;
    float price = 4;
}
```

Core Concepts: Protocol Buffers (protobuf)

Similar to a database schema or data contract, the **message format is predefined**.

Client and server already know what to expect, **eliminating the need for redundant data** in each message.

```
syntax = "proto3";

service RestaurantService {
    rpc GetEntrees (EntreeRequest) returns (EntreeResponse);
}

message EntreeRequest {
    int32 restaurant_id = 1;
}

message EntreeResponse {
    repeated Entree entrees = 1;
}

message Entree {
    int32 entree_id = 1;
    string name = 2;
    string description = 3;
    float price = 4;
}
```

Core Concepts: Unary gRPC (client-side)

As a result:

- **Ease of use for client**
 - RPCs look like a local function call
- **Compact and efficient**
 - Fewer bytes per message (over JSON)
 - Faster serialization / deserialization
- **Strongly typed**
 - Fewer errors, improved data integrity

```
EntreeResponse response = stub.getEntrees(request);

for (Entree entree : response.getEntreesList()) {
    System.out.println("Entree ID: " + entree.getEntreeId());
    System.out.println("Name: " + entree.getName());
    System.out.println("Description: " + entree.getDescription());
    System.out.println("Price: " + entree.getPrice());
    System.out.println("-----");
}
```

getEntrees(), getEntreesList()
are generated by protoc

Core Concepts: gRPC != protobuf

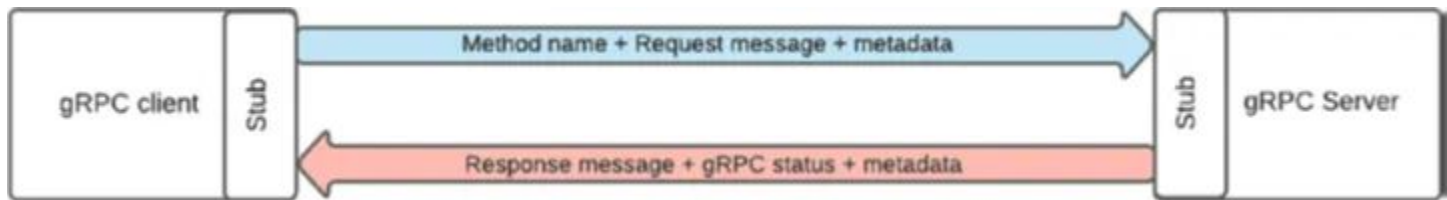
gRPC is primarily a **service-to-service** framework

- Libraries are available for most popular languages
- Browser support is available, but limited.
- **Can use protobuf without gRPC**
 - protoc handles message serialization / deserialization, code generation
 - gRPC (or something else, e.g. GCP Pub/Sub) handles network communication

Core Concepts: gRPC Service Methods

Unary RPCs (unidirectional)

- Client sends a **single request**
- Receives a **single response**, just like a normal function call

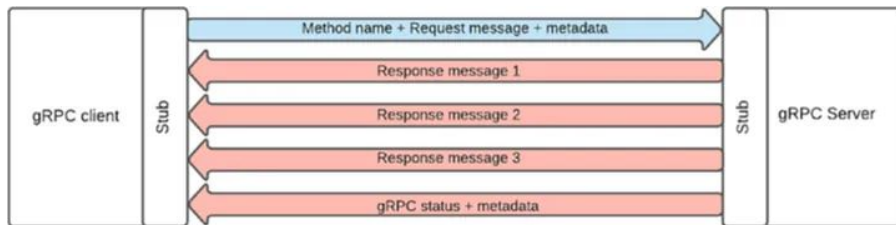


```
rpc SayHello(HelloRequest) returns (HelloResponse);
```

Core Concepts: gRPC Service Methods

Server streaming RPCs

- Use cases: real-time data feeds (stock prices), audio / video streaming
- **Client sends a request, receives a stream** to read a sequence of messages from server



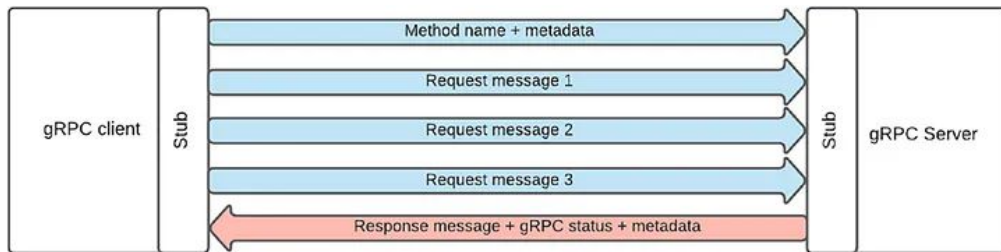
```
rpc LotsOfReplies(HelloRequest) returns (stream HelloResponse);
```

Source: [What Is gRPC and Different Types of gRPC Services](#)

Core Concepts: gRPC Service Methods

Client streaming RPCs

- Use cases: sensor data streaming, file upload, etc.
- **Client sends a stream** of messages, then waits
- Server receives and returns its response



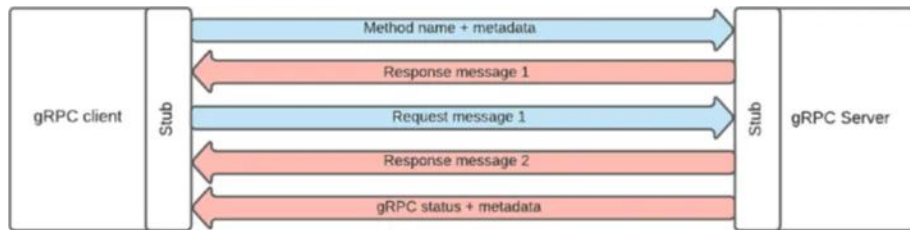
```
rpc LotsOfReplies>HelloRequest) returns (stream HelloResponse);
```

Source: [What Is gRPC and Different Types of gRPC Services](#)

Core Concepts: gRPC Service Methods

Bidirectional streaming RPCs

- Use cases: real-time chat, multiplayer gaming, video conferencing
- **Client and server can read and write at any time**
- Again, gRPC guarantees message ordering within an individual RPC call

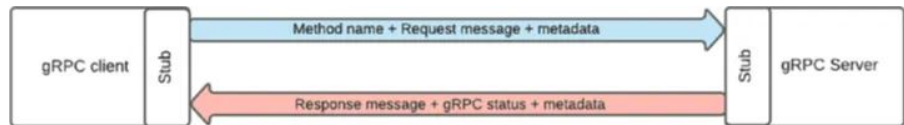


```
rpc BidiHello(stream HelloRequest) returns (stream HelloResponse);
```

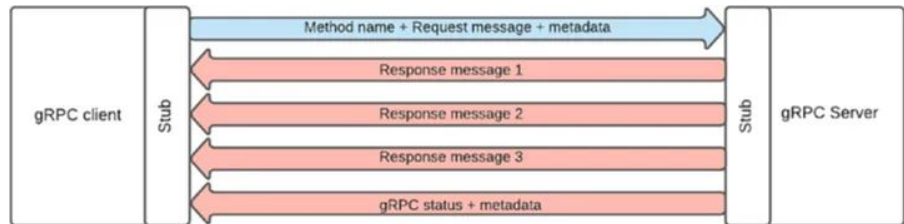
Source: [What Is gRPC and Different Types of gRPC Services](#)

Live Demo

Unary RPC



Server streaming RPC



Closing: Key Takeaways

- gRPC can be a promising option for **high-traffic, high-throughput** systems
- In my opinion, gRPC adds **unnecessary overhead** for simple and straightforward use cases



kinda, but not really :)

Closing: Key Takeaways

REST is still **very widely used** and more appropriate in a lot of cases

- Easier to understand and debug
- Preferable if **complex, hierarchical** data structures are necessary



kinda, but not really :)

Closing: Key Takeaways

Can also consider GraphQL

- Similar to REST, uses JSON over HTTP
- Clients specify what data they need, reducing over- or under-fetching



kinda, but not really :)

Resources

- [gRPC docs](#)
- Hello World examples
 - [Java](#)
 - [C++](#)
- [gRPC Keynote Presentation](#) at gRPC Conf 2023
- Learn how different teams use gRPC (and why)
 - [Uber's Next Gen Push Platform on gRPC](#), 2022
 - [Reddit's move to gRPC](#), 2021
 - [Adopting gRPC at Spotify](#), 2019